

Lab 2: ASP.NET Core Web API with Swagger

Software Required

- Visual Studio 2022 Community (Purple Icon)
 - .NET 8 SDK
 - Postman (Later)
-

Step 1: Open Visual Studio

1. Open **Visual Studio 2022 Community**.
 2. Click **Create a new project**.
-

Step 2: Select Project Template

In the search box, type:

ASP.NET Core Web API

Select:

ASP.NET Core Web API

Click **Next**.

Step 3: Configure Your Project

Project Name:

SwaggerDemo

Location:

Choose any folder.

Solution Name:

SwaggerDemo

Click **Next**.

Step 4: Additional Information

Select the following:

Option	Value
Framework	.NET 8.0 (Long Term Support)
Authentication Type	None
Configure for HTTPS	✓
Enable OpenAPI Support	✓
Use Controllers	✓
Native AOT	X

Click **Create**.

Step 5: Project Structure

Visual Studio creates the following files:

SwaggerDemo

```
|
|  └─ Controllers
|      └─ WeatherForecastController.cs
|
|  └─ Properties
|      └─ launchSettings.json
|
|  └─ appsettings.json
|
|  └─ Program.cs
|
|  └─ WeatherForecast.cs
|
└─ SwaggerDemo.csproj
```

Step 6: Run the Project

Press:

Ctrl + F5

The first time, you may see:

Trust ASP.NET Core SSL Certificate?

Click:

Yes

A browser opens:

<https://localhost:xxxx/swagger>

You should see the Swagger page.

If Swagger opens, your project is correct.

Step 7: Delete Default Files

Delete:

Controllers

WeatherForecastController.cs

Delete:

WeatherForecast.cs

Step 8: Create Employee Controller

Right-click:

Controllers

↓

Add

↓

Class

Name:

EmployeeController.cs

Click **Add**.

Step 9: Write Employee Controller

Replace everything with this code.

```
using Microsoft.AspNetCore.Mvc;

namespace SwaggerDemo.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeeController : ControllerBase
    {
        static List<string> employees = new List<string>()
        {
            "John",
            "David",
            "Smith"
        };

        [HttpGet]
        public IActionResult Get()
        {
            return Ok(employees);
        }

        [HttpPost]
        public IActionResult Post(string name)
        {
            employees.Add(name);
            return Ok("Employee Added Successfully");
        }

        [HttpPut]
        public IActionResult Put(int id, string name)
```

```

        {
            employees[id] = name;
            return Ok("Employee Updated Successfully");
        }

        [HttpDelete]
        public IActionResult Delete(int id)
        {
            employees.RemoveAt(id);
            return Ok("Employee Deleted Successfully");
        }
    }
}

```

Save the file.

Step 10: Program.cs

Keep **Program.cs** exactly like this.

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container.
builder.Services.AddControllers();
```

```
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen();
```

```
var app = builder.Build();
```

```
// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();

    app.UseSwaggerUI();
}

```

```
app.UseHttpsRedirection();
```

```
app.UseAuthorization();
```

```
app.MapControllers();
```

```
app.Run();
```

Do not add any other code yet.

Step 11: Build the Project

Click:

Build



Build Solution

Shortcut:

Ctrl + Shift + B

You should see:

Build succeeded.

Step 12: Run the Project

Press:

Ctrl + F5

Browser opens:

<https://localhost:xxxx/swagger>

You should see:

Employee

GET

POST
PUT
DELETE

Step 13: Test GET

Click:

GET



Try it out



Execute

Output:

```
[  
  "John",  
  "David",  
  "Smith"  
]
```

Status:

200 OK

Step 14: Test POST

Click:

POST



Try it out

Enter:

James



Execute

Output:

Employee Added Successfully

Step 15: Test PUT

Input:

id = 1

name = Robert

Output:

Employee Updated Successfully

Step 16: Test DELETE

Input:

id = 2

Output:

Employee Deleted Successfully

Step 17: Install Postman

1. Download and install Postman.
2. Open Postman.
3. Click **New → HTTP Request**.
4. Choose **GET**.
5. Enter the URL:

https://localhost:xxxx/api/employee

6. Click **Send**.

You should receive:

```
[  
  "John",  
  "David",  
  "Smith"  
]
```

Status:

200 OK

Step 18: Change Route

Open EmployeeController.cs.

Change:

```
[Route("api/[controller]")]
```

to:

```
[Route("api/Emp")]
```

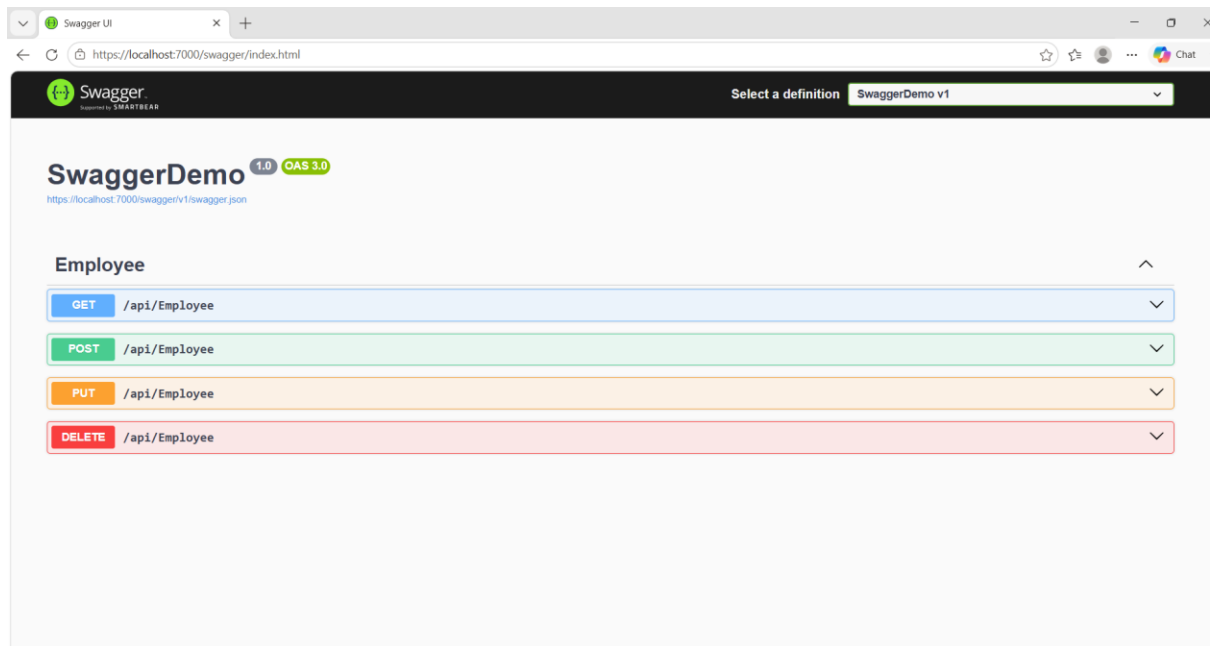
Save the file.

Run the project again.

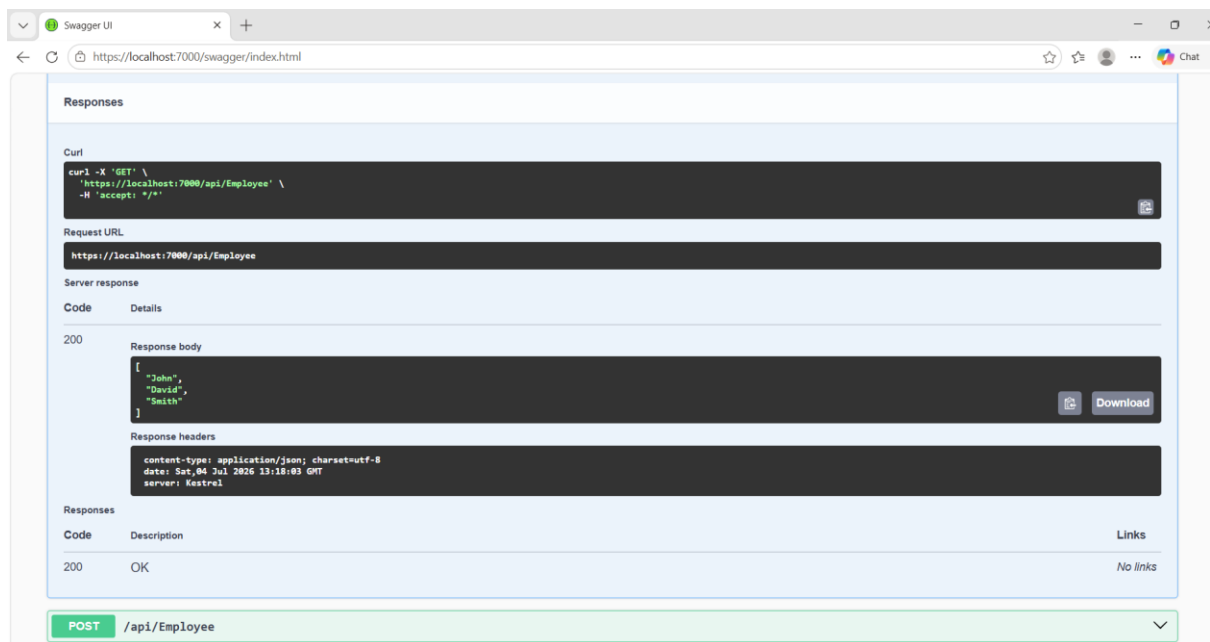
Now use this URL in Swagger or Postman:

<https://localhost:xxxx/api/Emp>

Outputs:



GET METHOD:



POST METHOD:

Swagger UI

https://localhost:7000/swagger/index.html

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7000/api/Employee?name=James' \
  -H 'accept: */*' \
  -d ''
```

Request URL

https://localhost:7000/api/Employee?name=James

Server response

Code	Details
200	Response body Employee Added Successfully Response headers <pre>content-type: text/plain; charset=utf-8 date: Sat, 04 Jul 2026 13:20:46 GMT server: Kestrel</pre>

Responses

Code	Description	Links
200	OK	No links

PUT /api/Employee

POST METHOD:

Swagger UI

https://localhost:7000/swagger/index.html

Responses

Curl

```
curl -X 'PUT' \
  'https://localhost:7000/api/Employee?id=1&name=Robert' \
  -H 'accept: */*' \
  -d ''
```

Request URL

https://localhost:7000/api/Employee?id=1&name=Robert

Server response

Code	Details
200	Response body Employee Updated Successfully Response headers <pre>content-type: text/plain; charset=utf-8 date: Sat, 04 Jul 2026 13:22:16 GMT server: Kestrel</pre>

Responses

Code	Description	Links
200	OK	No links

DELETE METHOD:

Swagger UI

https://localhost:7000/swagger/index.html

Chat

Parameters

Cancel

Name	Description
id	
integer(\$int32)	2
(query)	

ExecuteClear

Responses

Curl

curl -X 'DELETE' \ 'https://localhost:7000/api/Employee?id=2' \ -H 'accept: */*'

Request URL

https://localhost:7000/api/Employee?id=2

Server response

Code	Details
200	Response body Employee Deleted Successfully Download